



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS**

**“REAL-TIME SIGN LANGUAGE TRANSLATOR FOR DEAF-MUTE
COMMUNICATION”**

BY

HITESH LABH (49213)

MAUSAM LAMICHHANE (49218)

PRAYAG KHAREL (49224)

SUGAM SUBEDI (49240)

**A PROJECT SUBMITTED TO THE DEPARTMENT OF
ELECTRONICS AND COMPUTER ENGINEERING IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE
BACHELOR’S DEGREE IN ELECTRONICS COMMUNICATION AND
INFORMATION ENGINEERING**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS**

DHARAN, NEPAL

MARCH, 2026

REAL-TIME SIGN LANGUAGE TRANSLATOR FOR DEAF-MUTE COMMUNICATION

By

Hitesh Labh (49213)

Mausam Lamichhane (49218)

Prayag Kharel (49224)

Sugam Subedi (49240)

Project Supervisor

Asst. Prof. Manoj Kumar Guragai

A project submitted to the Department of Electronics and Computer Engineering in partial fulfillment of the requirements for the Bachelor's Degree in Electronics, Communication and Information Engineering.

Department of Electronics and Computer Engineering
Purwanchal Campus, Institute of Engineering
Tribhuvan University
Dharan, Nepal

March, 2026

COPYRIGHT©

This report is submitted as part of the fulfillment of the requirements for the Bachelor's Degree in Electronics, Communication and Information Engineering at Purwanchal Campus, Institute of Engineering, Tribhuvan University. The authors reserve academic rights over the contents of this report. Permission for scholarly use, limited reproduction, or reference may be granted with proper acknowledgment of the authors, supervisor, and the Department of Electronics and Computer Engineering, Purwanchal Campus.

Any copying, publication, or other use of this report for financial gain without prior written permission from both the department and the authors is prohibited.

Requests for permission to copy or otherwise use material from this report, in whole or in part, should be addressed to:

Head of Department

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering

Dharan, Sunsari

Nepal

DECLARATION

We declare that the work presented in this major project entitled “**Real-Time Sign Language Translator for Deaf-Mute Communication**” is our own original work carried out under the supervision of Asst. Prof. Manoj Kumar Guragai. This report has not been submitted, either in whole or in part, to any other university or institution for any academic award.

We authorize Purwanchal Campus, Institute of Engineering, Tribhuvan University to lend this report to other institutions or individuals for the purpose of scholarly research with proper acknowledgment.

Hitesh Labh (49213)

Mausam Lamichhane (49218)

Prayag Kharel (49224)

Sugam Subedi (49240)

March, 2026

ACKNOWLEDGEMENT

We are grateful to the Department of Electronics and Computer Engineering, Purwanchal Campus, Dharan, for providing us with the opportunity to carry out this project, which we believe will greatly contribute to our knowledge and future career in this field.

We would like to express our sincere gratitude to our supervisor **Asst. Professor. Manoj Kumar Guragai**, also the Head of the Department of Electronics and Computer Engineering, for his valuable support, leadership, and guidance throughout the development of this project. We are very thankful for his supervision in our project for his continuous encouragement, expert advice, patience and constructive feedback, which have been instrumental in successful completion of this project.

We also extend our heartfelt thanks to all the teachers of the Electronics and Computer Engineering Department who helped us directly and indirectly through their guidance, suggestions, and academic support during the course of this project. Their knowledge and encouragement have greatly contributed to our work.

Furthermore, we would like to thank our classmates and colleagues for their support and cooperation. Finally, we express our appreciation to the authors and publishers of various research materials that we referred to, which provided valuable insights and assistance in completing this project successfully.

ABSTRACT

Sign language is an important means of communication for deaf and mute individuals, but it is not widely understood by the general public. This project presents a real-time Android-based sign language translator that detects gestures from a live camera feed and converts them into readable text. The system uses two recognition modes: letter mode for static alphabet signs and word mode for dynamic signs. It is developed using Kotlin, Jetpack Compose, CameraX, MediaPipe Tasks, and TensorFlow Lite. For word recognition, pose and hand landmarks are converted into normalized temporal features for sequence-based inference. The project also addressed practical challenges such as mirroring errors, tensor-shape mismatches, UI redesign, frame-rate tuning, and random live predictions through iterative development. An optional ESP32-based display pathway was also explored for external output. The final system demonstrates the feasibility of a portable, low-cost sign language translator and provides a strong base for future improvement.

Keywords: *Sign language, TensorFlow Lite, MediaPipe, CameraX, ESP32.*

TABLE OF CONTENTS

COPYRIGHT©	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
1 INTRODUCTION	1
1.1 Background	1
1.2 Gap Identification	2
1.3 Motivation	3
1.4 Problem Statement	4
1.5 Objectives	4
1.6 Scope and Application	5
2 LITERATURE REVIEW	6
3 RELATED THEORY	8
3.1 Design Overview	8
3.2 Requirements Analysis	9
3.2.1 Functional Requirements	9
3.2.2 Non-Functional Requirements	9
3.3 Use-Case View	9
3.4 Overall Architecture	10

3.5	Hybrid Recognition Design	10
3.5.1	Reason for Separation	10
3.5.2	Mode Comparison	11
3.6	Hardware and Software Used	11
3.6.1	Hardware Used	11
3.6.2	Software Used	13
3.6.3	Tools and Components	13
3.7	Software Architecture	13
3.8	Data-Flow Design	14
3.9	Hardware Integration Design	14
3.10	System Constraints	16
4	METHODOLOGY	17
4.1	Development Overview	17
4.2	Recognition Workflow	19
4.3	Data Sources and Label Organization	19
4.4	Landmark Extraction Method	20
4.5	Feature Extraction	20
4.6	Clip Capture and Resampling	21
4.7	Model Preparation for Deployment	21
4.8	Android Development Method	22
4.9	Implementation	23
4.10	Performance-Tuning Methodology	24
4.11	Validation Methodology	24
5	RESULTS AND DISCUSSION	25

5.1	Nature of the Final Results	25
5.2	Final System Behavior	25
5.3	Letter Model Results	25
5.4	Word Model Results	27
5.5	Observed Testing Behavior	30
5.6	Engineering Challenges and Fixes	31
5.7	Discussion of Random Word Predictions	32
5.8	Effect of UI and Performance Tuning	33
5.9	Discussion of Hardware Integration	33
5.10	Comparison with the Original Objectives	33
5.11	Significance of the Result	33
6	CONCLUSION AND LIMITATIONS	34
6.1	Conclusion	34
6.2	Limitations	35
7	FUTURE ENHANCEMENT	36
	REFERENCES	37

LIST OF FIGURES

Figure 3.1: Block diagram of development	8
Figure 3.2: Overall system architecture of the final prototype	10
Figure 3.3: ESP32 development board used for hardware integration	12
Figure 3.4: OLED display module used for output visualization	12
Figure 3.5: Hybrid recognition pipeline used in the final system	14
Figure 3.6: Hardware integration architecture for external display support	15
Figure 4.1: Overall workflow of the hybrid sign-language recognition system	18
Figure 4.2: Letter dataset classes and train-validation split used for model preparation	19
Figure 4.3: Word-sequence train-test split with 30-step, 516-feature representation . .	20
Figure 4.4: BiLSTM-based word model architecture used for dynamic sign recognition	22
Figure 5.1: Letter-model training log excerpt showing convergence and validation performance	26
Figure 5.2: Training and validation accuracy for the letter-recognition model	26
Figure 5.3: Training and validation loss for the letter-recognition model	26
Figure 5.4: Confusion matrix of the final letter-recognition model	27
Figure 5.5: Training and validation accuracy for the word-recognition model	28
Figure 5.6: Training and validation loss for the word-recognition model	28
Figure 5.7: Word-model training log showing learning-rate reduction and early stopping	29
Figure 5.8: Word-model classification excerpt with Top-1, Top-3, and Top-5 accuracy	30

LIST OF TABLES

Table 3.1: Final recognition modes used in the project 11

Table 3.2: Hardware and software used 13

Table 5.1: Qualitative testing observations from the final system 31

Table 5.2: Observed implementation issues and the corresponding fixes 32

LIST OF ABBREVIATIONS

AI	: Artificial Intelligence
API	: Application Programming Interface
ASL	: American Sign Language
CNN	: Convolutional Neural Network
CV	: Computer Vision
ESP32	: Espressif 32-bit Microcontroller with Wi-Fi/Bluetooth
FPS	: Frames Per Second
IDE	: Integrated Development Environment
IoT	: Internet of Things
ML	: Machine Learning
OCR	: Optical Character Recognition
OLED	: Organic Light Emitting Diode
RNN	: Recurrent Neural Network
SDK	: Software Development Kit
SLR	: Sign Language Recognition
TFLite	: TensorFlow Lite
TTS	: Text To Speech
UI	: User Interface
Wi-Fi	: Wireless Fidelity

CHAPTER 1

INTRODUCTION

1.1 Background

Communication barriers still remain a serious social and practical problem for people who depend on sign language in daily life. A deaf or mute user may be able to communicate naturally through sign, but the person on the other side of the conversation may not understand those gestures. As a result, everyday interactions in hospitals, classrooms, public-service offices, shops, transportation systems, and personal settings can become slow, stressful, and dependent on the availability of a human interpreter.

Recent developments in mobile computer vision and on-device machine learning have made it possible to rethink this problem. Modern smartphones now provide high-quality cameras, optimized image-processing pipelines, hardware acceleration, and application frameworks that support real-time analysis. In parallel, frameworks such as MediaPipe and TensorFlow Lite have reduced the engineering cost of building systems that detect hand or body landmarks and run machine-learning models locally on mobile devices. These developments create an opportunity to design a portable sign-language translator that does not depend on bulky hardware or continuous internet connectivity.

The project described in this report was developed with this opportunity in mind. Rather than building a purely theoretical classifier, the work aimed to produce a realistic Android prototype that captures signs from a live front camera, processes them in real time, and displays the translated output in an accessible form. Over the course of development, the project evolved from an initial static-letter prototype into a hybrid system that supports both letter-level and word-level recognition. This evolution is central to the final report, because it reflects the real engineering path of the project: data preparation, feature design, model conversion, Android integration, debugging, and user-interface improvement were all shaped by what worked and what failed in practice.

In the final system, two different recognition paths are used because static letters and dynamic words do not behave in the same way. Letter recognition depends mainly on hand shape, so the system uses normalized single-hand landmarks for that task. Word recognition, however, re-

quires temporal understanding, since many signs involve movement and coordination between the hands and upper body. For this reason, the project uses a sequence-based pipeline in which pose and hand landmarks are collected over time, organized into a structured feature representation, and passed to a TensorFlow Lite word-recognition model. This distinction between static and dynamic recognition became one of the key design principles of the project.

Another important aspect of the project is that it was designed as a mobile engineering system rather than only a training experiment. The Android implementation integrates CameraX for live image acquisition, MediaPipe Tasks for landmark extraction, Jetpack Compose for the interface, and TensorFlow Lite for on-device inference. During development, several practical problems had to be addressed, including front-camera mirroring, handedness inconsistency, dependency-version mismatch, tensor-shape compatibility, unstable temporal predictions, and interface clarity during live use. Solving these issues was essential to make the prototype functional in a real device environment.

The project also considered accessibility beyond the mobile screen itself. In addition to showing recognized text inside the Android application, the system was designed with an optional external hardware pathway using ESP32 and OLED/TFT display modules. This extension allows the recognized output to be forwarded to a lightweight external display system, which makes the project more flexible for assistive use and demonstration purposes.

Therefore, the background of this work is not limited to the general need for sign-language translation. It also includes the practical challenge of transforming computer-vision and machine-learning methods into a portable, responsive, and understandable application that can operate in real time on consumer mobile hardware. The final prototype presented in this report is an attempt to bridge that gap by combining recognition accuracy, mobile feasibility, engineering compatibility, and user-oriented feedback in a single system.

1.2 Gap Identification

Although sign-language recognition has received significant research attention, several gaps remain when such systems are brought into practical mobile deployment. Large-scale research efforts such as MS-ASL and WLASL have significantly advanced benchmark-driven sign recognition, but their primary focus is on dataset creation, model comparison, and recognition performance rather than full mobile deployment constraints [3, 4]. Likewise, MediaPipe provides

an efficient framework for real-time perception and landmark tracking, yet it does not by itself solve application-level problems such as unstable live input, front-camera mirroring, user feedback, and end-to-end mobile interaction design [2, 8].

First, many research-oriented systems prioritize recognition performance on curated datasets but do not fully address the realities of live camera input, frame instability, changing lighting, front-camera mirroring, user feedback, and device-side latency [3, 4, 2]. Second, many systems focus mainly on either static hand signs or dynamic word-level recognition, but not both within a lightweight and unified mobile design [3, 4]. Third, practical deployment on portable devices remains a challenge because real-time camera integration, on-device inference, and efficient execution must all be handled together; tools such as CameraX and TensorFlow Lite make this possible, but the engineering gap between model development and usable deployment is still substantial [5, 6]. Fourth, accessibility-oriented interaction design is often treated as secondary, even though practical assistive systems need clear status feedback, understandable outputs, and optional multimodal interaction such as speech support [7, 5].

The present project addresses these gaps by emphasizing *mobile feasibility*, *hybrid recognition*, *engineering compatibility*, and *user-facing feedback*. The final implementation targets Android, uses on-device inference, and separates static and dynamic signs into recognition modes more suitable for their nature [5, 6, 8]. It also explicitly documents the engineering obstacles encountered during development, including model-version mismatches, handedness errors caused by camera mirroring, and unstable word prediction due to weak sign segmentation, thereby addressing issues that are often underemphasized in benchmark-focused literature [2, 3, 4].

1.3 Motivation

The main motivation for this project is social relevance. A sign-language translator can improve inclusivity by helping bridge communication between deaf or mute individuals and people who do not know sign language. The project is also technically motivating because it sits at the intersection of computer vision, machine learning, Android engineering, and embedded-system integration.

From an educational perspective, the project created an opportunity to work with a complete product-style pipeline rather than a single algorithm. The team had to understand how to collect and organize recognition labels, how to represent gestures numerically, how to prepare models

for TensorFlow Lite, how to integrate them in Android Studio, how to debug real-time inference, and how to think about a user interface that communicates state clearly. This made the project not only an accessibility application but also a comprehensive engineering exercise.

1.4 Problem Statement

How can a low-cost, portable, and reasonably responsive sign-language translator be built on Android so that static letter signs and selected dynamic word signs can be recognized from a live front-camera feed and translated into human-readable output without requiring external computation for the main inference loop?

This problem contains multiple sub-problems:

- obtaining a feature representation that is compact enough for mobile inference yet expressive enough to capture sign structure,
- distinguishing between static and dynamic signs within one application,
- designing a word-level capture and preprocessing pipeline that can fit the expectations of a sequence model,
- maintaining compatibility between training-side preprocessing and Android-side inference,
- producing useful user feedback in the presence of uncertainty or missing detections, and
- exploring a hardware output path for external display without overloading the mobile application.

1.5 Objectives

- To develop a real-time Android-based sign language recognition system that supports both static letter recognition and dynamic word recognition using on-device inference.
- To integrate the recognition system with a user-friendly interface and an optional ESP32-based external display pathway for practical accessibility support.

1.6 Scope and Application

This project focuses on the development of a prototype-level real-time sign language recognition system for Android rather than a full commercial translator. Its scope includes Android application development using Kotlin and Jetpack Compose, live camera input through CameraX, landmark extraction using MediaPipe Tasks, on-device inference with TensorFlow Lite, static letter recognition, dynamic word recognition, reverse fingerspelling mode, user-interface improvement, and an optional external output pathway through ESP32-based display hardware.

The project does not aim to provide full continuous-sentence translation, universal sign-language coverage, or large-scale signer-independent evaluation. Instead, it demonstrates a practical and portable engineering foundation for assistive communication. The system is mainly applicable to educational demonstrations, controlled sign-language interaction, prototype accessibility tools, and future expansion into mobile and hardware-assisted communication support.

CHAPTER 2

LITERATURE REVIEW

Research in sign-language recognition spans classical image processing, handcrafted feature extraction, deep learning, and multimodal perception. Earlier systems often relied on gloves, colored markers, or controlled laboratory conditions, but more recent work focuses on camera-based recognition that can operate with minimal intrusion on the user. With the rise of deep learning, the most common recognition paradigms now include convolution-based image classification, keypoint- or landmark-based modeling, and temporal sequence analysis for dynamic gestures.

A key distinction in the literature is the difference between static-sign recognition and dynamic-sign recognition. Static recognition treats each sign as a single posture or stable hand shape, which is suitable for alphabet recognition, menu gestures, or short symbolic commands. Dynamic recognition, by contrast, must preserve motion, timing, and inter-frame relationships. Many sign-language words are not identifiable from a single image; they depend on movement trajectory, pose transition, and repetition patterns. This distinction strongly influenced the design of the present project.

A major trend in modern sign-language recognition is the use of landmarks instead of raw pixels. Landmark-based systems reduce input dimensionality and can be more robust to background clutter. Instead of asking a model to learn directly from a full image, a detector first converts the image into a structured representation such as hand keypoints or upper-body pose keypoints. This representation can then be normalized and classified.

Sign language has long been recognized as a complete visual communication system with its own linguistic structure and grammar. One of the foundational works in this area is by Stokoe, who established sign language as a structured and meaningful language rather than a simple collection of gestures [1].

With the advancement of computer vision, landmark-based perception frameworks have become increasingly useful in gesture and sign recognition systems. MediaPipe provides a flexible framework for building real-time perception pipelines and has been widely adopted for hand and pose tracking tasks [2].

Large-scale datasets have also played an important role in sign-language recognition research. The MS-ASL dataset introduced by Joze and Koller provides a large benchmark for understanding American Sign Language and has supported the development of deep-learning methods for sign recognition [3].

Similarly, Li et al. presented the WLASL dataset for word-level sign-language recognition, enabling comparative study of recognition methods on large-vocabulary sign datasets [4].

These datasets demonstrate the growing importance of data-driven approaches in modern sign-language research. For mobile implementation, Android development tools such as CameraX and TensorFlow Lite provide an effective pathway for real-time deployment. CameraX simplifies camera integration on Android devices and supports practical live-image acquisition [5].

TensorFlow Lite enables optimized on-device machine-learning inference, making it suitable for portable sign-language translation systems [6].

In addition, Android TextToSpeech support offers a useful option for converting recognized signs into speech output, improving accessibility and interaction [7].

Its efficiency and suitability for real-time applications make it highly relevant for mobile sign-language systems. In addition, MediaPipe Tasks Vision documentation has supported practical integration of hand and pose landmark detection into deployable applications [8].

Deep-learning techniques have become increasingly important in modern computer-vision and sequence-recognition systems. According to Chollet, neural-network architectures are highly effective for extracting meaningful patterns from complex visual and temporal data. These methods are widely applied in image classification, gesture recognition, and mobile AI applications, making them relevant to sign-language recognition systems.[9].

Goodfellow, Bengio, and Courville explain that deep learning enables hierarchical feature extraction from raw input data, allowing models to learn increasingly abstract representations. Their work highlights the importance of sequence modeling, normalization, and generalization in machine-learning systems, which are directly relevant to temporal sign-language recognition.[10]

CHAPTER 3

RELATED THEORY

3.1 Design Overview

The system design was developed progressively from a static hand-sign recognizer into a hybrid mobile translator that supports both static letters and dynamic words. Figure 3.1 summarizes the major development flow from camera acquisition to inference and output.

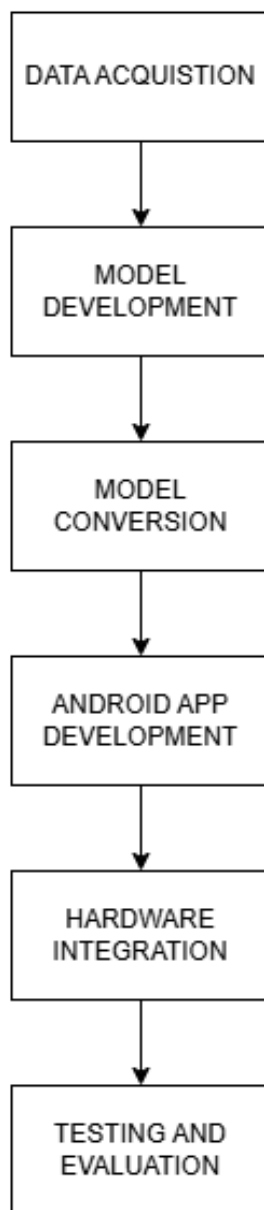


Figure 3.1: Block diagram of development

3.2 Requirements Analysis

3.2.1 Functional Requirements

The major functional requirements of the system include live camera operation, dual recognition modes, real-time prediction, user interaction support, and output display. The application shall open the front camera and provide a live preview to the user. It shall support two operational modes: letter recognition and word recognition. In letter mode, the application shall detect a hand and produce a predicted letter or control token. In word mode, the application shall automatically capture a short temporal segment, preprocess it, and run a word classifier. The user interface shall display the current mode, system state, and prediction text. The system shall also allow the user to clear captured word data and switch between recognition modes. In addition, the project should remain extendable to external output devices such as an ESP32-driven display.

3.2.2 Non-Functional Requirements

Important non-functional requirements of the system include portability, responsiveness, interpretability, maintainability, and extensibility. The system should be portable enough to run on an Android device without depending on a server for the main recognition loop. It should also be responsive so that status changes and predictions appear quickly enough for real-time use and live demonstration. Interpretability is important because users must clearly understand whether the system is idle, detecting, recording, or predicting at any given time. Maintainability is also necessary so that the code remains understandable and can support iterative debugging and future improvements. In addition, the system should be extensible enough to support later model replacement, larger vocabularies, and optional hardware-output integration.

3.3 Use-Case View

The basic user workflow begins when the user opens the Android application and the front-camera preview becomes available. The user then selects either letter mode or word mode, presents the sign in front of the camera, and waits for the application to display the predicted output. Depending on the mode, the prediction may be shown immediately or after a short word-

capture interval. The recognized result can remain on the phone interface and may optionally be spoken or forwarded to an external display.

3.4 Overall Architecture

Figure 3.2 shows the overall high-level design of the final system. The figure highlights the major processing stages and the optional hardware-output pathway.

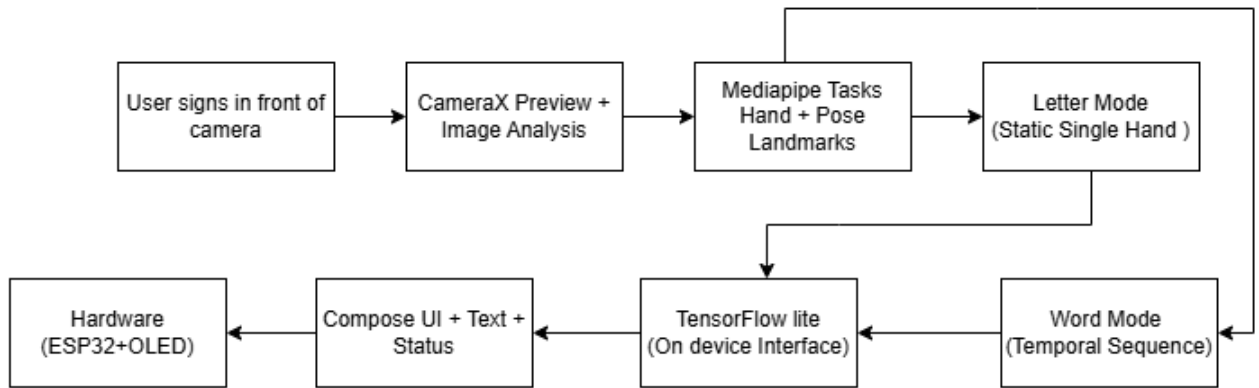


Figure 3.2: Overall system architecture of the final prototype

The architecture reflects four core design decisions: landmark extraction is shared by both recognition modes, letter and word recognition are separated after landmark acquisition, the main inference loop remains on the Android device, and hardware output is treated as a downstream extension rather than a core dependency.

3.5 Hybrid Recognition Design

A central system-design choice was the adoption of a **hybrid recognition strategy**. Letters and words are not forced into one single model path because their temporal properties are different.

3.5.1 Reason for Separation

The reason for separating the two recognition modes is based on the nature of the signs themselves. Letters are mostly static and can be recognized effectively through single-frame hand-shape analysis, whereas words are dynamic and require temporal modeling across multiple frames along with pose context. A unified model would have increased system complexity

and reduced control over runtime behavior. By separating the modes, the project is able to use simpler preprocessing for letter recognition and a more expressive temporal pipeline for word recognition.

3.5.2 Mode Comparison

Table 3.1: Final recognition modes used in the project

Mode	Input Representation	Temporal Nature	Primary Use
Letter mode	One hand, 21 landmarks, normalized to a 63-value feature vector	Mostly static	Alphabet signs and control tokens such as <code>space</code> , <code>del</code> , and <code>nothing</code>
Word mode	Pose plus both hands, 258 raw values per frame, expanded to 516 features after preprocessing	Dynamic sequence	Selected isolated words that depend on motion and posture over time

3.6 Hardware and Software Used

This project integrates mobile computer vision, embedded hardware, and machine learning deployment to build a real-time sign language recognition and display system. The overall implementation combines an Android-based recognition application with external display hardware for output visualization.

3.6.1 Hardware Used

The hardware components used in this project are the Android smartphone camera, an ESP32 development board, and an OLED display module. The front camera of the Android phone serves as the primary acquisition device and captures the user's hand and upper-body gestures in real time. The ESP32 development board was used as the external hardware controller for displaying recognized text output. It provides built-in Wi-Fi and enough processing capability to receive stable text from the Android application and forward it to the connected display. A compact OLED display module was used to visualize the recognized letters or words sent from

the Android application through the ESP32.

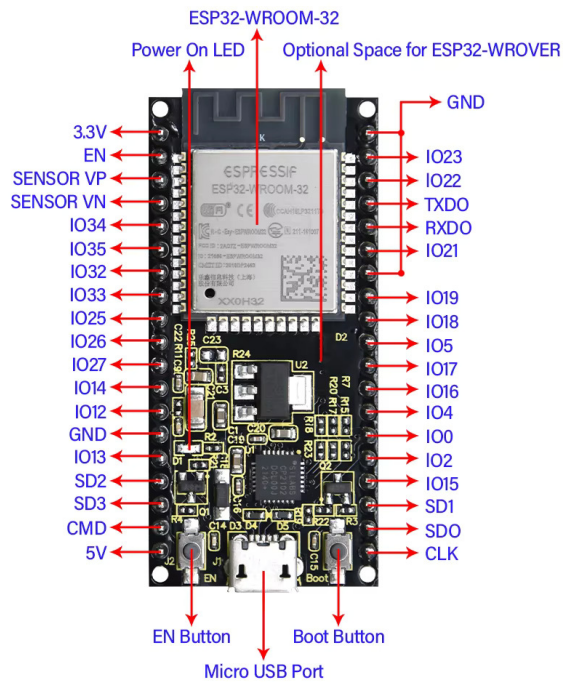


Figure 3.3: ESP32 development board used for hardware integration
Source: AliExpress

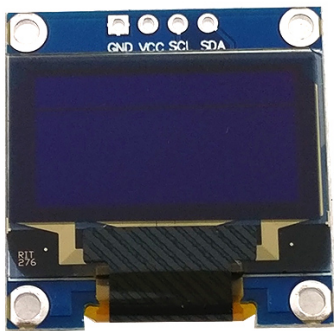


Figure 3.4: OLED display module used for output visualization
Source: Shot from Phone

3.6.2 Software Used

The software stack of the project consists of Android Studio, Kotlin with Jetpack Compose, CameraX, MediaPipe Tasks, TensorFlow Lite, Arduino IDE, and Git/GitHub. Android Studio was used as the main development environment. CameraX handled live preview and image analysis, MediaPipe Tasks provided hand and pose landmark extraction, and TensorFlow Lite was used for on-device inference of both the letter and word models. Arduino IDE was used for programming the ESP32 module and testing external display behavior, while Git and GitHub were used for version control and project backup.

3.6.3 Tools and Components

Table 3.2: Hardware and software used

Category	Component / Tool	Purpose
Hardware	ESP32 Development Board	Receives recognized output and controls display module
Hardware	OLED Display Module	Displays recognized letters or words
Hardware	Android Smartphone Camera	Captures real-time sign gestures
Software	Android Studio	Develops and runs the Android application
Software	Kotlin + Jetpack Compose	Implements application logic and user interface
Software	CameraX	Captures and processes camera frames
Software	MediaPipe	Extracts hand and pose landmarks
Software	TensorFlow Lite	Runs sign recognition models on-device
Software	Arduino IDE	Programs ESP32 and OLED output logic
Software	Git / GitHub	Version control and project management

3.7 Software Architecture

At the software level, the final Android design can be understood as a stack of cooperating layers: a Compose-based user-interface layer, a CameraX preview and analysis layer, a MediaPipe perception layer, a TensorFlow Lite inference layer, and a feedback layer for on-screen text,

status updates, optional speech output, and optional external display support.

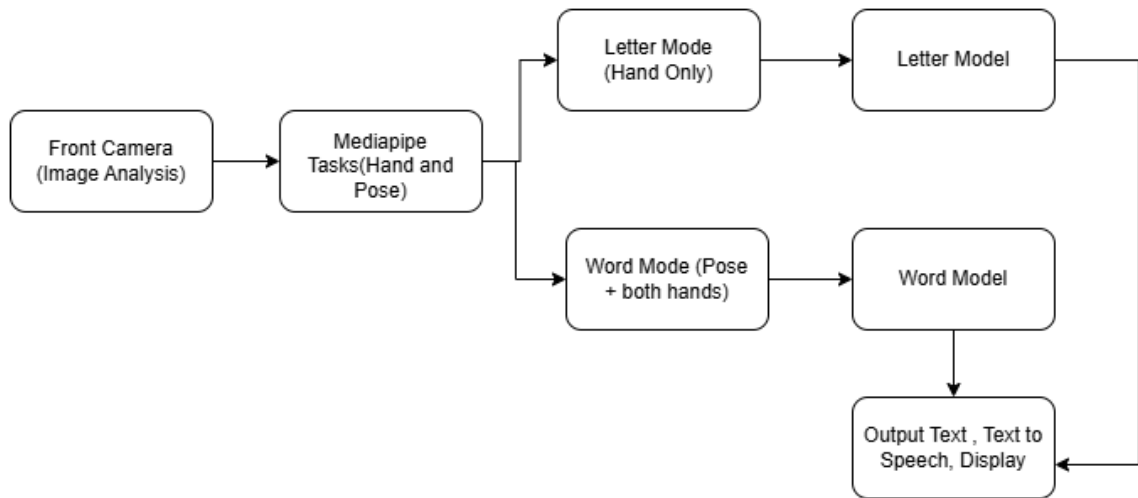


Figure 3.5: Hybrid recognition pipeline used in the final system

3.8 Data-Flow Design

The internal data flow differs by mode. In letter mode, the application captures the current frame, detects at least one hand, extracts 21 hand landmarks, normalizes the hand around the wrist, mirrors the normalized X values when needed, sends the resulting 63-value feature vector to the letter model, and displays the predicted class in the interface. In word mode, the application captures a live frame stream, detects pose and one or two hands, builds a 258-value raw feature vector per frame, continues capture while hands remain active, stops after the clip is complete or after a small number of missing-hand frames, resamples the sequence to 30 time steps, adds velocity to create 516 features per time step, and then runs the word model.

3.9 Hardware Integration Design

Although the core translator runs entirely on Android, the project also explored how recognized text could be shared with external hardware. The most practical design was to send only the final stable prediction rather than the full camera stream. This keeps the mobile application responsible for the heavy inference and lets the hardware extension focus on user feedback.

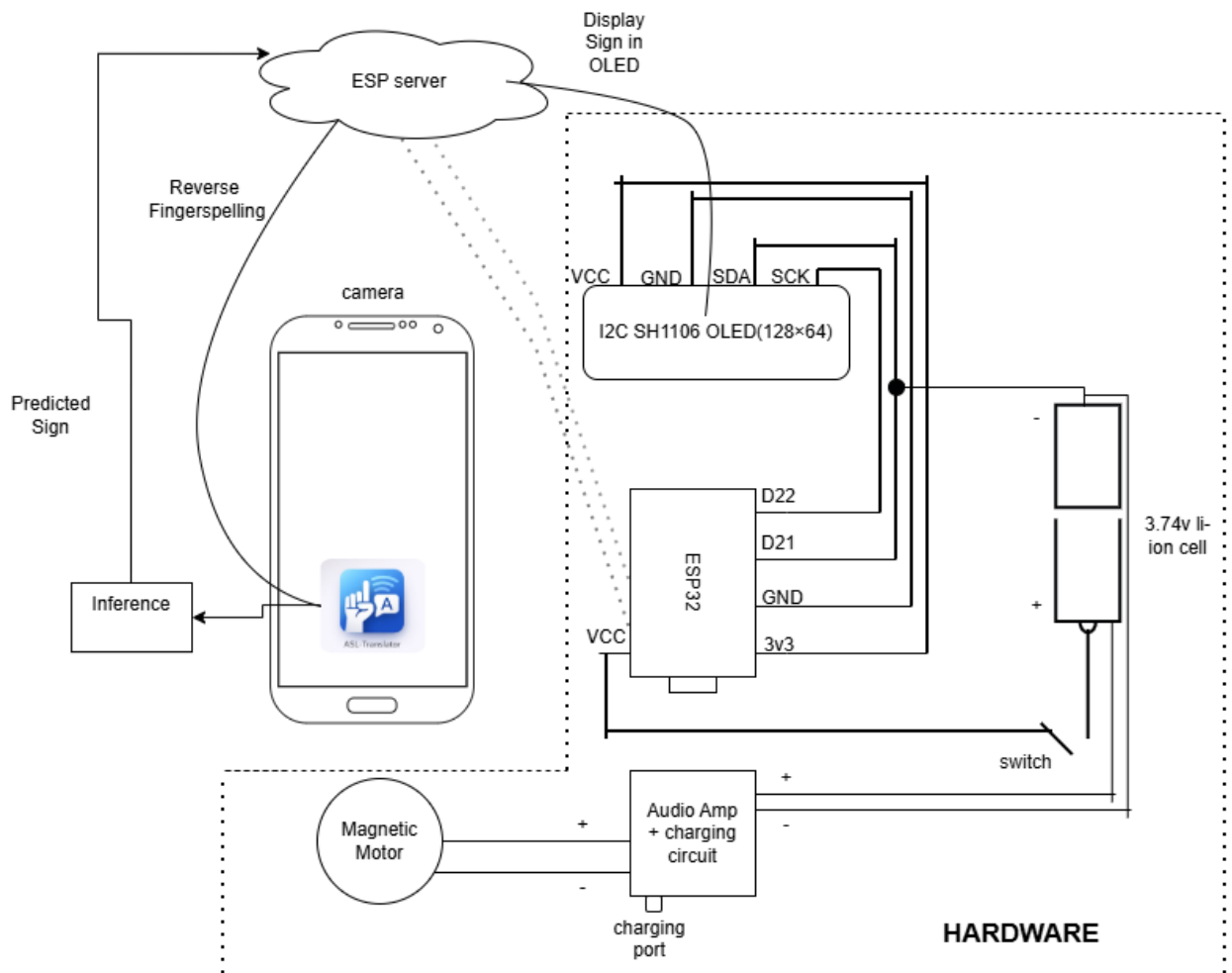


Figure 3.6: Hardware integration architecture for external display support

The figure shows how the hardware components and software are integrated together that makes a proposed sign language translator app. The process starts when a user signs in front of android camera which is captured in real time. The application system decides wheather it is static sign or a sign with a motion and finds out wheather the sign is letter or word and runs the inference on the input sign (either letter or word model). The model gives output in the form of text which can be seen in the application User Interface in real time. Side by Side the ouput is fed to ESP server hosted by ESP32 which is captured by ESP32 which is connected to OLED display where the output is displayed. The output is also delivered as speech played in speaker powered by Li-ion battery which also powers the ESP32. The system also allows reverse fingerspelling i.e. text which is given as input to app and that is converted to the corresponding fingerspelling sign of ASL.

3.10 System Constraints

The design was constrained by several practical factors, including mobile-device memory and processing power, variable lighting and background conditions, front-camera orientation issues, incomplete segmentation of signs in live capture, the risk of mismatch between training-side and Android-side preprocessing, and limited time available for fully polishing the end-to-end hardware pathway.

The developed system operates under several practical constraints related to mobile-device processing, camera behavior, and machine-learning inference. The application relies heavily on real-time landmark extraction using the device camera, making prediction quality dependent on camera stability, frame rate, and environmental visibility conditions.

The word-recognition pipeline also requires fixed-length temporal sequences, meaning that inconsistent sign duration or abrupt motion can reduce recognition reliability. Furthermore, because TensorFlow Lite inference is performed locally on the Android device, memory usage and CPU performance impose additional runtime constraints, especially during longer sessions.

The system currently assumes front-camera usage and single-user interaction. Multi-user recognition, background activity separation, and continuous conversational signing remain outside the scope of the current implementation.

CHAPTER 4

METHODOLOGY

4.1 Development Overview

The project followed an iterative development methodology rather than a strictly linear sequence. It began with a static letter-recognition prototype based on hand landmarks and later evolved into a hybrid Android system capable of both letter-level and word-level sign recognition. As the project progressed, the scope expanded from simple landmark-based classification to temporal word recognition, Android deployment, interface redesign, and hardware-assisted output. This iterative process was necessary because the final system required repeated refinement of preprocessing, model compatibility, runtime behavior, and user interaction.

The overall workflow of the final system is shown in Figure 4.1. The application starts by initializing the Android app and camera pipeline. From there, recognition proceeds through two branches: a letter-recognition path for static signs and a word-recognition path for dynamic temporal sequences. Both branches ultimately produce user-visible output and support optional output pathways such as speech and external display modules.

Static letters sign are fed to letter model that detects static hand from the frame captures the hand landmarks only and normalizes it and then gives input to the model and the model recognizes the letter which is displayed as text in the display unit and also the corresponding sound for the letter is generated. Dynamic positions of hand and arms denote the word sign. Thus to detect the word, hands and pose landmarks are extracted over certain frames in real time and are given as input to the model in a way it accepts. Finally, the word model runs inference on input and gives out the recognized word in text displayed in display unit. Also the sound for the corresponding word is generated via text to speech.

In this way, input are separated along two paths which are separately treated by their own models so that the output of the overall system remains more accurate.

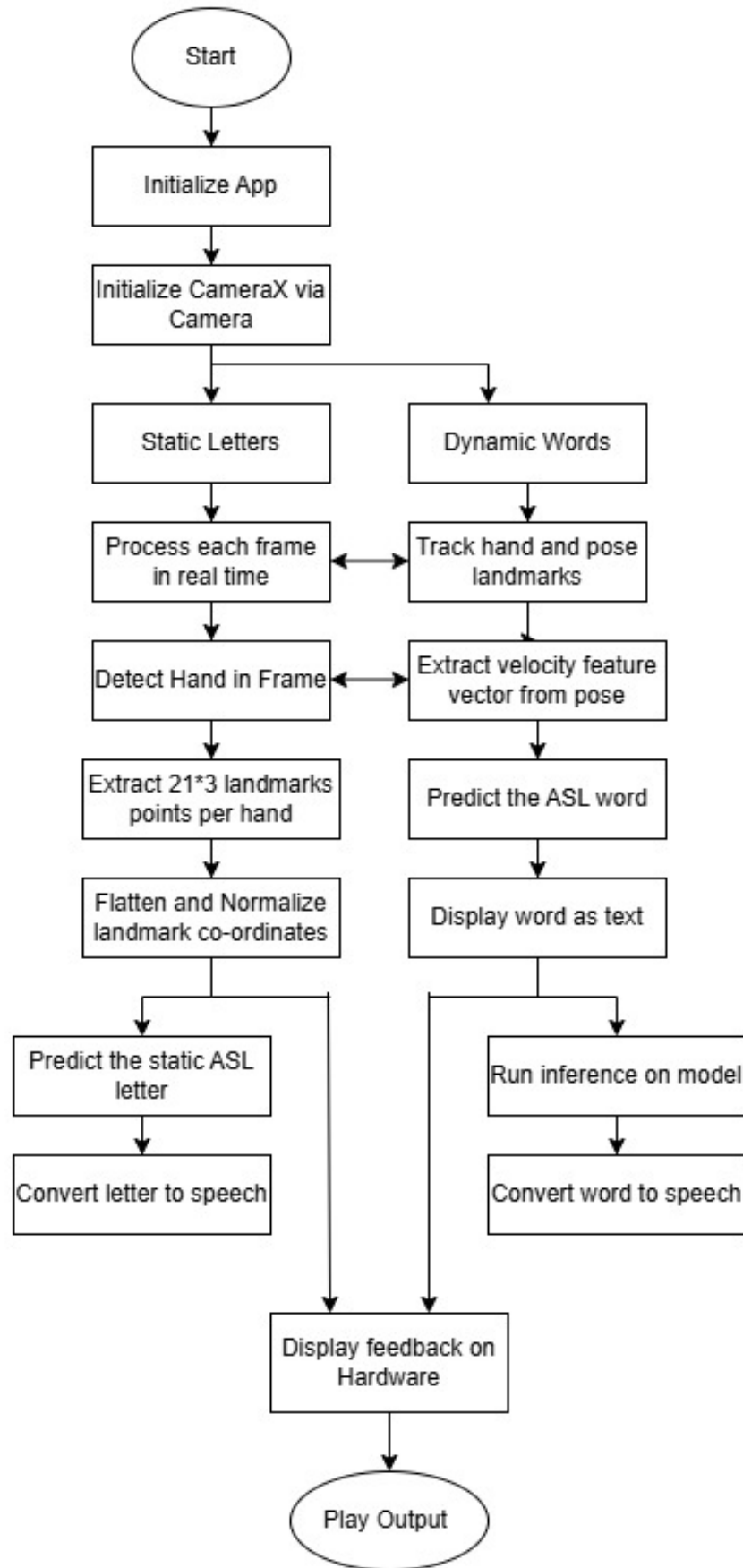


Figure 4.1: Overall workflow of the hybrid sign-language recognition system

4.2 Recognition Workflow

The translator follows a live processing workflow consisting of camera capture, landmark extraction, mode-specific preprocessing, TensorFlow Lite inference, and user-visible feedback. Although both recognition modes run within the same Android application, they use different feature representations because static alphabet signs and dynamic word signs require different modeling strategies.

4.3 Data Sources and Label Organization

The project used sign-language data organized separately for static letters and dynamic words. The final deployed application does not depend on a single uniform dataset size; instead, the label sets and training assets were structured according to the needs of the deployed models.

For the static letter model, the dataset contains alphabet signs along with special control classes such as `del` and `space`. Figure 4.2 shows the encoded classes, number of classes, and the train-validation split used during letter-model preparation. This confirms that the training representation for the letter model consisted of normalized hand landmarks with shape (21,3).

```
Encoded classes: ['A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R'
'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z' 'del' 'space']
Number of classes: 28
Train: (57305, 21, 3) (57305, 28)
Val: (6368, 21, 3) (6368, 28)
Augmented train: (171915, 21, 3) (171915, 28)
```

Figure 4.2: Letter dataset classes and train-validation split used for model preparation

For word recognition, the system uses a sequence-based dataset that stores temporal features across 30 time steps. The deployed word-label asset contains 54 classes corresponding to commonly used sign words. ["ACCIDENT", "APPLE", "BED", "BEFORE", "BLACK", "BLUE", "CAN", "CHAIR", "CLOSE", "COLD", "COLOR", "COOL", "COUSIN", "COW", "DANCE", "DARK", "DEAF", "ENJOY", "FAMILY", "FINE", "FINISH", "FISH", "FORGET", "GO", "GOOD", "HAPPY", "HELP", "HOT", "KISS", "LAST", "LATER", "LIKE", "MAKE", "MAN", "MANY", "MORE", "NEED", "NO", "PLAY", "PLEASE", "RUN", "SHIRT", "SICK", "SMALL", "TABLE", "TALL", "TEACH", "WAIT", "WHITE", "WRITE", "YES", "YESTERDAY", "YOU", "YOUR"] Figure 4.3 shows the train-test split used for the temporal word model, where each sample has the shape (30,516).

```
X_train, X_test, y_train, y_test, y_train_int, y_test_int = train_test_split(
    X,
    y,
    y_int,
    test_size=TEST_SIZE,
    random_state=SEED,
    stratify=y_int
)

print("Train:", (variable) X_test: Any hape)
print("Test :", X_test.shape, y_test.shape)
```

```
Train: (1080, 30, 516) (1080, 54)
Test : (270, 30, 516) (270, 54)
```

Figure 4.3: Word-sequence train-test split with 30-step, 516-feature representation

4.4 Landmark Extraction Method

Landmark extraction was performed using MediaPipe Tasks. Two detectors were used in the final system: HandLandmarker for hand tracking and PoseLandmarker for body pose extraction. Hand landmarks were required in both modes, whereas pose landmarks were especially important for dynamic word recognition. The hand pipeline detects up to two hands, and the pose pipeline detects one person. This design allowed the project to maintain a lightweight mobile implementation while still supporting dynamic sign modeling.

In the word-recognition branch, the pose component contributes 33 landmarks and each hand contributes 21 landmarks. The project also revealed an important engineering issue during development: even when training and Android deployment use the same model, differences in feature ordering, pose-channel handling, and handedness assumptions can significantly affect performance. Documenting the landmark structure therefore became an important part of the methodology.

4.5 Feature Extraction

The word-recognition pipeline constructs a raw feature vector of 258 values per frame. This vector consists of 132 pose values, 63 left-hand values, and 63 right-hand values. After extraction, the pose and hand components are normalized separately to reduce the effect of camera position, distance, and hand size.

For letter recognition, preprocessing is simpler. A single hand is selected, the wrist is used as

the reference point, and all landmarks are normalized relative to this wrist-centered coordinate system. If the detected hand corresponds to the left hand, the normalized X-coordinates are mirrored so that the model sees a more consistent orientation.

For word recognition, normalization is followed by temporal enrichment. Velocity is computed as the frame-to-frame difference,

$$v_t = x_t - x_{t-1} \quad (4.1)$$

which doubles the effective representation from 258 to 516 values per time step:

$$516 = 258 + 258.$$

This temporal representation was one of the key refinements that made the Android-side pre-processing compatible with the deployed sequence model.

4.6 Clip Capture and Resampling

In a live mobile environment, the application does not receive perfectly segmented sign clips. Therefore, the Android implementation includes runtime capture logic before word inference. A captured word clip is built from a short sequence of frames and terminated either when the required number of frames has been gathered or when the hand disappears for a small number of consecutive frames after a minimum valid capture has been reached.

Because the sequence model expects 30 time steps, captured clips are resampled to a fixed length before inference. This allows variable live motion to be aligned with the model’s expected input format. In the final hybrid implementation, the capture size, minimum early-stop threshold, and prediction-hold duration were tuned experimentally to support stable real-time testing.

4.7 Model Preparation for Deployment

The deployment methodology emphasized compatibility between training-time assumptions and Android runtime behavior. This was especially important because multiple model variants and asset names existed during the development process. Earlier stages of the project used legacy files, while later stages used updated compatible models for hybrid Android deployment.

For the static letter-recognition model, training showed fast convergence and stable validation performance. Figure 5.2 and Figure 5.3 show the training and validation curves used during letter-model preparation. These figures indicate that the model reached high validation accuracy while maintaining low validation loss.

For word recognition, a sequence model with BiLSTM layers was deployed. The compatible word model accepts an input sequence of shape (30,516) and outputs probabilities across 54 word classes. The architecture summary used in the project is shown in Figure 4.4.

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 30, 516)	0
bidirectional (Bidirectional)	(None, 30, 256)	660,480
bidirectional_1 (Bidirectional)	(None, 128)	164,352
dense (Dense)	(None, 192)	24,768
dropout (Dropout)	(None, 192)	0
dense_1 (Dense)	(None, 96)	18,528
dropout_1 (Dropout)	(None, 96)	0
dense_2 (Dense)	(None, 54)	5,238

Total params: 873,366 (3.33 MB)
Trainable params: 873,366 (3.33 MB)
Non-trainable params: 0 (0.00 B)

Figure 4.4: BiLSTM-based word model architecture used for dynamic sign recognition

4.8 Android Development Method

The application was developed in Kotlin using Android Studio. CameraX was used for live preview and image analysis, Jetpack Compose was used for the interface, MediaPipe Tasks was used for landmark extraction, and TensorFlow Lite was used for on-device inference. The final Android design supports separate letter and word modes, analyzer-side sequence handling,

state-based user feedback, optional speech output, and an optional hardware bridge for external display.

4.9 Implementation

The final prototype was implemented as a native Android application using Kotlin and Jetpack Compose. CameraX was used to open the front camera and provide both preview and analysis streams. The analyzer branch passed frames to MediaPipe Tasks for landmark extraction and then routed the extracted features to the appropriate TensorFlow Lite model depending on the selected mode. In letter mode, the application processes a single detected hand at a time. The 21 hand landmarks are normalized around the wrist, optionally mirrored when handedness indicates the left hand, and flattened into a 63-value feature vector. This vector is then passed to the deployed static letter model, and the predicted letter or control token is shown immediately in the interface.

In word mode, the application captures a short temporal segment rather than making a prediction from a single frame. Pose and hand landmarks are combined into a 258-value raw feature vector for each frame. The captured sequence is then normalized, resampled to 30 time steps, enriched with velocity information to form 516 features per time step, and finally sent to the BiLSTM-based word model. Additional state logic was implemented to manage clip start, clip end, and prediction-hold behavior.

The user interface was redesigned using Jetpack Compose so that the application could clearly indicate the active mode, the current system status, and the most recent prediction. This redesign improved usability during live testing by making it easier to distinguish between idle, recording, and predicted states. The implementation also kept room for auxiliary outputs such as Android Text-to-Speech and the ESP32-based external display pathway. The external hardware pathway was implemented as an extension to the Android recognition system rather than as a separate inference platform. Stable predictions from the phone were transmitted to the ESP32 through a lightweight communication bridge, and the ESP32 forwarded the received text to an OLED display. This approach allowed the phone to remain responsible for all computationally intensive tasks while the external hardware handled only communication and output display.

4.10 Performance-Tuning Methodology

Performance tuning was necessary to make the app usable in real time. The main adjustments included reducing analysis overhead, using the `STRATEGY_KEEP_ONLY_LATEST` camera mode, lowering analysis resolution when necessary, and simplifying the UI so the user could clearly understand whether the app was idle, recording, or predicting. These changes improved responsiveness and made live testing more practical.

4.11 Validation Methodology

The final validation focused more on functional and qualitative testing than on claiming a single benchmark value for the completed hybrid Android application. This decision was appropriate because the final system combined multiple evolving components, and some early training artifacts did not fully reflect the final deployment pipeline.

Validation was therefore centered on whether the application launched correctly, whether camera frames were acquired, whether landmarks were extracted in both modes, whether preprocessing matched the deployed model structure, whether the application handled no-hand and active-sign cases correctly, and whether word capture produced plausible outputs under controlled signing conditions.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Nature of the Final Results

The final result of this project is a working Android prototype for hybrid sign-language recognition. The outcome is not limited to offline training; instead, it combines live camera input, landmark extraction, preprocessing, on-device TensorFlow Lite inference, interface feedback, and optional external display pathways. Because the project changed significantly over time, the results presented here emphasize the final deployed behavior while still including the most relevant training evidence for the final models.

5.2 Final System Behavior

The final application successfully demonstrates live preview through the Android camera, letter-mode recognition through hand landmarks, and word-mode recognition through temporal sequences built from pose and hand landmarks. It also communicates recognition states clearly through the redesigned interface, which made testing and debugging easier.

5.3 Letter Model Results

The static letter-recognition model produced strong performance during training. Figure 5.1 shows a training log excerpt, while Figure 5.2 and Figure 5.3 show the corresponding accuracy and loss curves. Together, these figures indicate that the model converged quickly and maintained strong validation behavior.

The letter model achieved a final training accuracy of approximately 93.92% and a validation accuracy of approximately 98.13%. The final training loss decreased to about 0.1915, while the validation loss decreased to about 0.0629. These values indicate that the model learned the hand-landmark patterns effectively and generalized well to unseen validation data. The dataset used for this model had the shape (63676, 21, 3), was organized into 28 classes, and was augmented to increase robustness during training.

```

Epoch 13/40
5373/5373 ————— 28s 5ms/step - accuracy: 0.9353 - loss: 0.2043 - val_accuracy: 0.9797 - val_loss: 0.0753
...
Epoch 16/40
5373/5373 ————— 24s 5ms/step - accuracy: 0.9375 - loss: 0.1940 - val_accuracy: 0.9812 - val_loss: 0.0650
Epoch 17/40
5373/5373 ————— 25s 5ms/step - accuracy: 0.9392 - loss: 0.1915 - val_accuracy: 0.9813 - val_loss: 0.0629

```

Figure 5.1: Letter-model training log excerpt showing convergence and validation performance

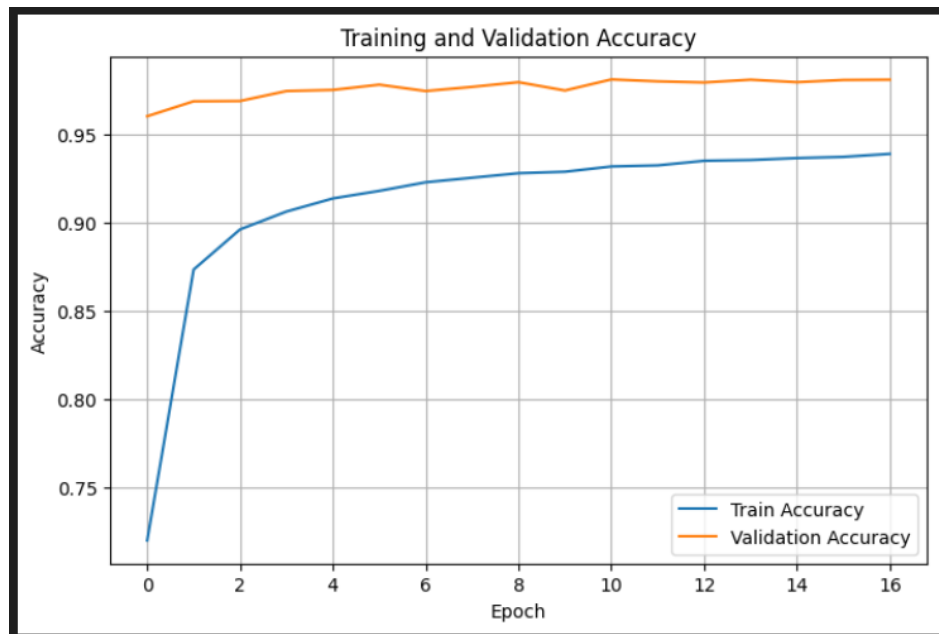


Figure 5.2: Training and validation accuracy for the letter-recognition model

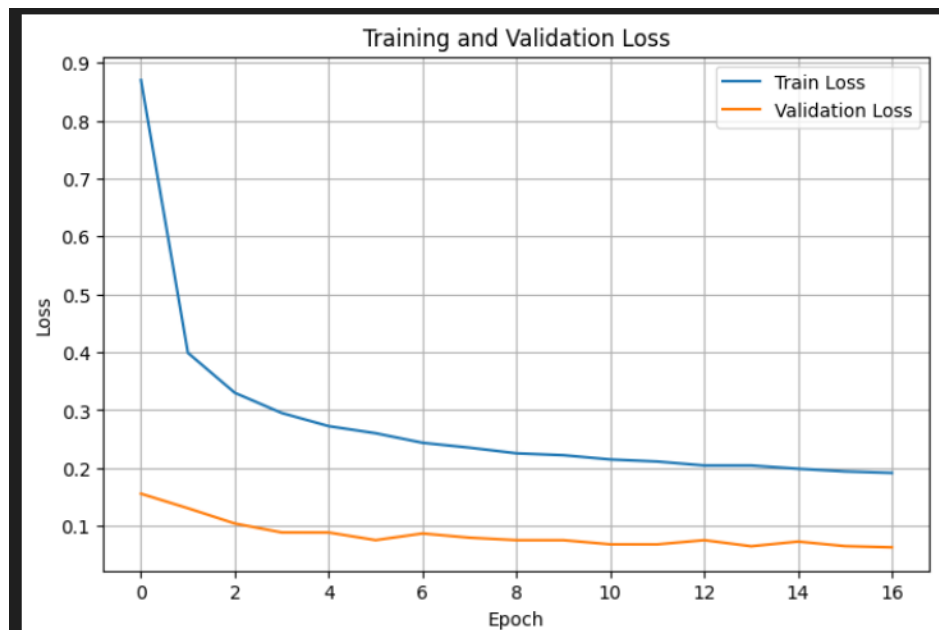


Figure 5.3: Training and validation loss for the letter-recognition model

A more detailed class-level view is provided in Figure 5.4. The confusion matrix shows that

most letter classes are recognized correctly, with only a small number of off-diagonal errors. This behavior supports the practical use of the letter model for controlled static-sign demonstrations.

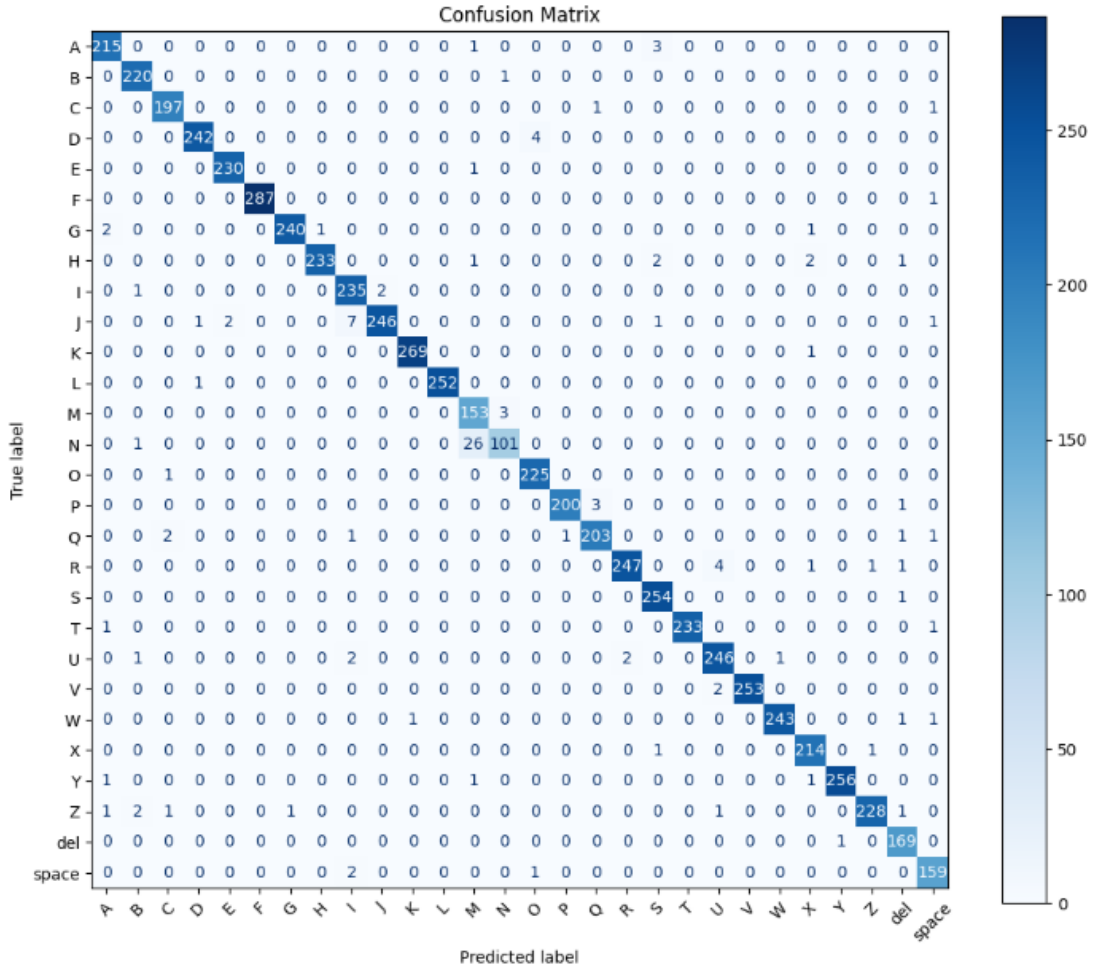


Figure 5.4: Confusion matrix of the final letter-recognition model

5.4 Word Model Results

The word-recognition model was more challenging because it depends on sequence quality, sign segmentation, and close alignment between training and Android-side preprocessing. Figure 5.5, Figure 5.6, and Figure 5.7 summarize the training behavior of the temporal model.

The word-recognition model produced moderate but promising performance, reflecting the higher complexity of dynamic sign recognition compared with static letter classification. The final training accuracy reached approximately 86.37%, while the best validation accuracy was approximately 68.15%. Near the final training stage, the validation accuracy remained in the range of about 67.04% to 67.41%. The training loss was about 1.0008 and the validation loss

was about 1.6568. These results suggest that the model was able to learn useful temporal sign patterns, although generalization remained more difficult than in the letter model. The sequence input had the shape (30,516), and the model architecture contained 873,366 trainable parameters.

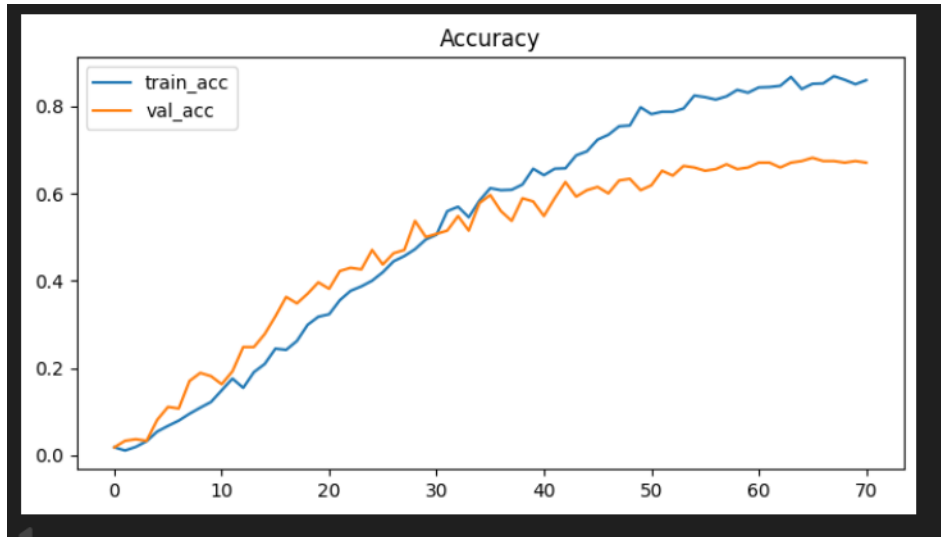


Figure 5.5: Training and validation accuracy for the word-recognition model

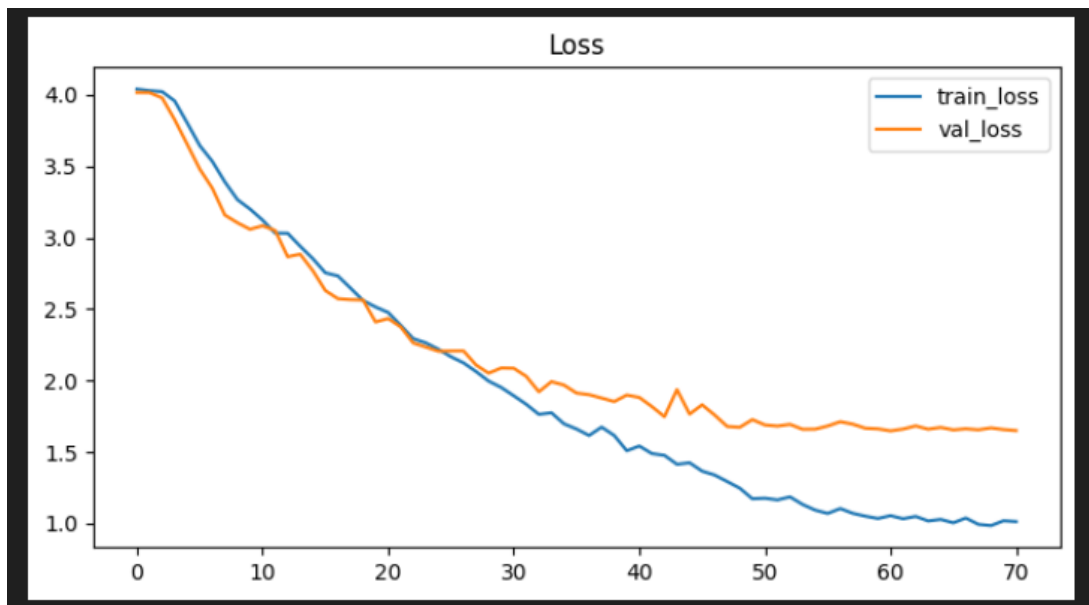


Figure 5.6: Training and validation loss for the word-recognition model


```

Epoch 69: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
[1m68/68][0m][32m-----][0m][37m][0m][1m3s][0m 49ms/step - accuracy: 0.8602 - loss: 0.9849 - top3_acc: 0.9769 - top5_acc: 0.9917 -
val_accuracy: 0.6704 - val_loss: 1.6685 - val_top3_acc: 0.8444 - val_top5_acc: 0.9148 - learning_rate: 6.2500e-05
Epoch 70/80
[1m67/68][0m][32m-----][0m][37m][0m][1m0s][0m 42ms/step - accuracy: 0.8634 - loss: 1.0008 - top3_acc: 0.9707 - top5_acc: 0.9893
Epoch 70: val_accuracy did not improve from 0.68148
[1m68/68][0m][32m-----][0m][37m][0m][1m3s][0m 46ms/step - accuracy: 0.8500 - loss: 1.0187 - top3_acc: 0.9704 - top5_acc: 0.9907 -
val_accuracy: 0.6741 - val_loss: 1.6568 - val_top3_acc: 0.8444 - val_top5_acc: 0.9148 - learning_rate: 3.1250e-05
Epoch 71/80
[1m67/68][0m][32m-----][0m][37m][0m][1m0s][0m 41ms/step - accuracy: 0.8637 - loss: 0.9999 - top3_acc: 0.9720 - top5_acc: 0.9893
Epoch 71: val_accuracy did not improve from 0.68148
[1m68/68][0m][32m-----][0m][37m][0m][1m3s][0m 45ms/step - accuracy: 0.8593 - loss: 1.0129 - top3_acc: 0.9731 - top5_acc: 0.9898 -
val_accuracy: 0.6704 - val_loss: 1.6499 - val_top3_acc: 0.8481 - val_top5_acc: 0.9148 - learning_rate: 3.1250e-05
Epoch 71: early stopping
Restoring model weights from the end of the best epoch: 61.

```

Figure 5.7: Word-model training log showing learning-rate reduction and early stopping

The temporal word task achieved lower validation accuracy than the letter model, which was expected because dynamic sign recognition is more complex and is more sensitive to segmentation quality. Figure 5.8 provides the classification excerpt together with Top-1, Top-3, and Top-5 accuracy. The Top-1 accuracy was 67.04%, while Top-3 and Top-5 accuracies reached 84.07% and 90.74%, respectively. This gap shows that the correct class was often contained within the higher-ranked predictions even when the first prediction was incorrect.

	precision	recall	f1-score	support
ACCIDENT	0.400	0.400	0.400	5
APPLE	1.000	1.000	1.000	5
BED	0.333	0.200	0.250	5
BEFORE	0.500	0.200	0.286	5
BLACK	1.000	0.600	0.750	5
BLUE	0.667	0.400	0.500	5
CAN	0.833	1.000	0.909	5
CHAIR	1.000	1.000	1.000	5
CLOSE	0.500	0.400	0.444	5
COLD	0.667	0.800	0.727	5
COLOR	0.714	1.000	0.833	5
COOL	1.000	0.400	0.571	5
COUSIN	0.455	1.000	0.625	5
COW	1.000	0.600	0.750	5
DANCE	0.800	0.800	0.800	5
DARK	0.800	0.800	0.800	5
DEAF	0.833	1.000	0.909	5
ENJOY	1.000	0.600	0.750	5
FAMILY	0.833	1.000	0.909	5
FINE	0.750	0.600	0.667	5
FINISH	1.000	0.800	0.889	5
FISH	0.429	0.600	0.500	5
FORGET	1.000	1.000	1.000	5
...				
Top-1 accuracy: 0.6704				
Top-3 accuracy: 0.8407				
Top-5 accuracy: 0.9074				

Figure 5.8: Word-model classification excerpt with Top-1, Top-3, and Top-5 accuracy

5.5 Observed Testing Behavior

Testing of the final Android prototype was performed under realistic development conditions, including different framing conditions, hand visibility changes, and repeated switching between recognition modes. Letter mode behaved more consistently when a single hand was clearly framed, whereas word mode performed best when the signer produced short isolated signs with clear beginning and ending motion.

Table 5.1: Qualitative testing observations from the final system

Test Aspect	Observed Behavior	Interpretation
Letter mode with one clear hand	Responsive and straightforward predictions in controlled framing	Suitable for static demonstration and alphabet-style interaction
Word mode with isolated short signs	Best behavior occurred when the signer clearly began and ended the sign	Confirms the importance of clip segmentation for temporal recognition
Rapid transitions between signs	Could trigger unstable or premature word predictions	Indicates the need for stronger sign-boundary detection and background rejection
No-hand situations	UI was able to return to waiting or ready states	Important for usability and debugging transparency
Front-camera behavior	Mirroring and handedness assumptions had strong impact on results	Confirms that camera-orientation handling is a critical part of the system
Longer sessions	Performance tuning improved responsiveness but did not eliminate all recognition variability	Suggests further optimization and model retraining opportunities

5.6 Engineering Challenges and Fixes

A major outcome of the project was the engineering knowledge gained from repeated integration failures and subsequent fixes. The final system improved only after multiple rounds of dependency correction, preprocessing alignment, runtime redesign, and interface refinement. These issues were especially important because the project combined several moving parts: landmark extraction, sequence preprocessing, model compatibility, and live Android execution.

Table 5.2: Observed implementation issues and the corresponding fixes

Issue	Observed Symptom	Fix or Mitigation
MediaPipe dependency mismatch	Runtime crashes and unstable landmark behavior	Aligning dependency versions in the Android build and using consistent task packages
Front-camera mirroring before detection	Left/right hand assignment became inconsistent and harmed recognition quality	Separating preview mirroring from inference assumptions and revisiting handedness handling
Model and input-shape mismatch	Random or meaningless word predictions	Reconstructing the expected 30-step, 516-feature word input pipeline on Android
Weak sign segmentation	Word model guessed classes from partial or transition frames	Adding clip capture logic, early-stop conditions, and prediction-hold behavior
Legacy asset-name mismatch	Labels or models could be loaded inconsistently across project stages	Documenting asset families and keeping runtime file names synchronized with code
Poor UI clarity	Harder to tell whether the app was idle, recording, or predicting	Redesigning the Compose UI with explicit state panels and mode-aware controls

5.7 Discussion of Random Word Predictions

The most persistent challenge in the project was incorrect or seemingly random word prediction. This issue emerged because temporal sign recognition depends not only on model quality but also on live sequence quality. In a real-time Android setting, the model may receive partial clips, transition frames, unstable handedness assignments, or temporally inconsistent inputs. Even a correct TensorFlow Lite model can therefore produce misleading outputs if the runtime preprocessing does not closely match the training pipeline.

The project did not eliminate this problem entirely, but it clarified its causes and introduced several mitigations. These included more explicit capture logic, stronger state handling, clearer separation between letter and word modes, and a more faithful reconstruction of the 30-step, 516-feature Android-side input pipeline expected by the sequence model.

5.8 Effect of UI and Performance Tuning

Another important result was the improvement in usability after redesigning the interface and tuning the runtime behavior. These changes may seem secondary from a purely offline machine-learning perspective, but they were highly important for practical testing. Clearer mode indicators, better status feedback, and improved frame-handling behavior made the system easier to demonstrate, interpret, and debug during live use.

5.9 Discussion of Hardware Integration

The hardware pathway explored in this project is best understood as an optional extension to the Android recognition engine rather than as the main inference platform. The project findings showed that Android is the appropriate place for the heavy vision and machine-learning operations, while the ESP32-based pathway is more suitable for receiving final recognized text and forwarding it to an external display device such as an OLED or TFT screen. This division of responsibility simplifies the overall architecture and makes the hardware module more practical.

5.10 Comparison with the Original Objectives

The project substantially achieved its original goals. An Android-based prototype was developed, letter recognition was successfully implemented, word recognition was achieved at prototype level, TensorFlow Lite was used for on-device inference, the user interface was improved, and a hardware-integration pathway was explored. Although the reliability of dynamic word recognition still requires refinement, the overall objectives were met to a meaningful engineering standard.

5.11 Significance of the Result

The most significant outcome of the project is that it progressed from concept to end-to-end working prototype. Rather than remaining an offline training exercise, it became a multi-component accessibility-oriented application that combines computer vision, mobile deployment, machine learning, interface design, and practical debugging. This makes the project a strong foundation for future work in mobile sign-language technology.

CHAPTER 6

CONCLUSION AND LIMITATIONS

6.1 Conclusion

This project set out to build a practical real-time sign-language translator for Android and to investigate how such a system could be deployed using modern mobile machine-learning tools. The final outcome is a hybrid Android prototype capable of recognizing both static letters and selected dynamic words from a live front-camera feed. The project integrates CameraX, MediaPipe Tasks, TensorFlow Lite, and a Compose-based user interface into a single end-to-end application.

A major contribution of the work is the documented transition from simple letter classification to a more sophisticated word-recognition pipeline. This transition required the design of a 258-value raw landmark representation, the use of per-component normalization, the construction of temporal clips, resampling to a fixed sequence length, and the addition of velocity features to create a 516-value input per time step. Just as importantly, the project documented the engineering challenges that arose during Android deployment, including dependency mismatches, handedness issues, asset-version mismatch, UI weaknesses, and unstable word predictions.

From an academic perspective, the project demonstrates that a student-led accessibility application can move meaningfully beyond isolated notebooks and into a real mobile prototype. From an engineering perspective, it shows that the success of sign-language translation depends not only on the model but also on feature design, runtime segmentation, compatibility, and user feedback.

The project also demonstrated the importance of consistency between training-time preprocessing and Android-side runtime inference. During development, several issues such as orientation mismatch, handedness inconsistency, sequence instability, and model-input mismatch significantly affected prediction quality. Resolving these problems provided valuable practical experience in mobile machine-learning deployment and highlighted that successful real-time recognition depends on both model quality and system integration.

6.2 Limitations

Although the final system is functional and valuable as a prototype, it has several limitations. The word recognition model currently supports only a limited vocabulary, focusing on a selected set of classes rather than a comprehensive sign-language lexicon. In addition, dynamic word recognition still relies on relatively clean and isolated signing, making it sensitive to transitions between signs.

The system is also affected by signer and environmental variability, where factors such as lighting conditions, camera angle, distance from the device, background clutter, and individual signing styles can significantly influence prediction accuracy. Furthermore, the project underwent multiple iterations of model and label updates during development, which introduced challenges in maintaining a fully standardized final configuration.

Another limitation lies in the maturity of external hardware integration. Although an ESP32-based display pathway was explored, it remains an auxiliary feature, and the Android application itself is the primary deliverable. Finally, the evaluation scope of the system is limited, as the project does not present a single definitive accuracy metric for the final hybrid system due to differences between development-stage experiments and the deployed pipeline.

The system currently performs best under relatively controlled conditions and may experience reduced accuracy when multiple people appear in the frame or when the hand is partially occluded. Dynamic environmental conditions such as rapid motion blur, inconsistent lighting, and low camera quality can also reduce landmark stability and negatively affect predictions.

Another limitation is computational dependency on mobile-device capability. Lower-end Android devices may experience reduced frame-processing speed, delayed inference, or temporary interface lag during continuous recognition tasks. Since the application performs real-time computer-vision operations on-device, performance may vary significantly across hardware platforms.

CHAPTER 7

FUTURE ENHANCEMENT

Future work on this project can significantly improve both the technical performance and the usability of the system. One important area of enhancement is sign segmentation. A more robust start and stop detection mechanism for dynamic word recognition should be developed. This could involve the use of a dedicated background class, motion-based gating strategies, or temporal confidence smoothing methods specifically designed for real-time applications.

Another key direction is the expansion and improvement of the dataset. Increasing the vocabulary size and collecting more diverse training data across multiple users, lighting environments, and camera setups would improve the generalization capability of the model and make performance evaluation more reliable.

Future research may also explore a unified yet adaptive inference architecture that can handle both static letter recognition and dynamic word recognition within a single model. While doing so, it will be important to preserve the interpretability and stability advantages provided by the current hybrid approach.

Enhancements can also be made to output modalities. The text-to-speech functionality can be further refined with better configurability and naturalness, while the ESP32-based external display system can be fully developed into a polished accessory for demonstrations, classroom use, and assistive communication scenarios.

User-centered improvements are equally important. Conducting structured user studies with individuals who rely on sign language would provide valuable feedback on system usability, interface clarity, response time, and overall trust in predictions. This feedback would guide future design improvements and ensure the system meets real-world accessibility needs.

Finally, deployment-level improvements should be prioritized to ensure robustness. These include packaging stable model and label combinations, implementing automated validation of input formats, introducing logging mechanisms for debugging difficult cases, and optimizing performance across different Android devices.

REFERENCES

- [1] W. C. Stokoe, *Sign Language Structure: An Outline of the Visual Communication Systems of the American Deaf*. Buffalo, NY: University of Buffalo, 1960.
- [2] C. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019.
- [3] S. Joze and O. Koller, “MS-ASL: A Large-Scale Data Set and Benchmark for Understanding American Sign Language,” in *Proceedings of the British Machine Vision Conference*, 2019.
- [4] D. Li, C. Rodriguez, X. Yu, and H. Li, “Word-Level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison,” in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision*, 2020.
- [5] Android Developers, “CameraX Overview,” Accessed for implementation guidance during the project.
- [6] TensorFlow, “TensorFlow Lite,” Accessed for mobile model deployment guidance during the project.
- [7] Android Developers, “TextToSpeech,” Accessed for speech-output integration guidance during the project.
- [8] Google AI Edge, “MediaPipe Tasks Vision Documentation,” Accessed for landmark-detection integration guidance during the project.
- [9] F. Chollet, “Deep Learning with Python.” 2nd ed. Shelter Island, NY: Manning Publications, 2021.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, “Deep Learning.” Cambridge, MA: MIT Press, 2016.